

Verified Program Synthesis for Coinductive Reinforcement Learning

From Observations to Optimal Policies via Predicate DSL

Ricardo Corral-Corral

February 2026

Abstract

We present a verified program synthesis framework that extends Coinductive Symmetric Homomorphism Reinforcement Learning (CSHRL) from learned rankings to *synthesized programs*. Where CSHRL defines optimality as structure preservation between action rankings and infinite reward streams, we add a synthesis layer that *constructs* these rankings automatically from environment observations.

The framework introduces ten components, all machine-verified in Agda: **(1)** A *Predicate DSL* (PredProg)—a domain-specific language for Boolean predicates over environment features, parameterized by an abstract carrier type and feature set, enabling instantiation across different Environment Classes; **(2)** A *CEGIS* (Counterexample-Guided Inductive Synthesis) algorithm that enumerates predicate programs and refines a version space by observations, with proven monotonic convergence and strict shrinkage bounds; **(3)** A *Propagation Theorem* establishing that feature-equivalent carriers are indistinguishable to any predicate program, which yields three quantified consequences: identical refinement (Refine-Equiv), observation subsumption, and exploration dissolution—reducing the observations needed from $|C|$ (carrier count) to $|C/\sim|$ (equivalence classes); **(4)** *Exploration completeness*: after observing one representative per equivalence class, no further observation can refine the version space (“dissolution is all you need”); **(5)** *Temporal propagation*: when transitions preserve feature equivalence, observations propagate along entire trajectories, not just across carriers—mirroring CSHRL’s coinductive unfolding; **(6)** *Feature sufficiency*: when the target function respects feature equivalence, every CEGIS survivor is provably correct on *all* carriers, not just the observed ones—upgrading stabilization to a full correctness guarantee; **(7)** *Multi-predicate propagation*: when synthesizing multiple predicates from the same feature space (e.g., *is-dead* and *is-solved*), observations are shared—one representative set suffices for all predicates, and cross-predicate entailment (e.g., $\text{dead} \rightarrow \neg \text{solved}$) further reduces evaluations; **(8)** *Online synthesis with learning integration*: observation order is provably irrelevant (refinement commutes), replayed observations are provably redundant, and a generic LearningBridge connects learning feedback to synthesis with a bridge sufficiency theorem composing learning convergence with synthesis correctness; **(9)** *Sample complexity lower bound*: two targets agreeing on observed carriers produce identical version spaces (same-obs-same-vs), so survivors transfer freely between indistinguishable targets (survivor-transfer); combined with the upper bound (sufficient-cegis), this proves the sample complexity is *exactly* $|C/\sim|$; **(10)** *DSL completeness*: a constructive fingerprint construction proves that the Boolean PredProg DSL is expressively complete for feature-respecting targets—for any target constant on equivalence classes, there exists a PredProg matching it everywhere (dsl-complete).

We demonstrate the full pipeline end-to-end on three Environment Classes: a Finite Deterministic MDP (1D Maze) where CEGIS synthesizes ranking predicates that yield a verified CoindHomo and optimal policy; a Combinatorial Placement MDP (N -Queens) where synthesized predicates *define* the environment’s step function, enabling find-policy to solve 4-Queens instantly and 8-Queens (with verified equivalence to hand-crafted predicates) including partial solution completion from any intermediate state; and a Stochastic Finite MDP where synthesis operates on expected traces, producing a verified StochasticCoindHomo—demonstrating that the entire generic theory applies unchanged to probabilistic transitions.

This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code while producing this PDF. The synthesis framework builds on the CSHRL foundation [1] without modifying it—all results are additive.

1 Introduction

CSHRL [1] establishes a new foundation for optimal sequential decision making: an agent is optimal when its ranking of actions forms a *coinductive symmetric homomorphism* into the ordering of infinite reward streams. Learning reduces to *restoring* the structural correspondence between rankings and outcomes via violation correction.

But how does a ranking come to exist in the first place?

In the original framework, rankings are either hand-crafted (verified manually) or discovered by the Environment Class `find-policy` (which exhaustively evaluates the state-action tree). Both approaches have limitations:

- **Hand-crafted rankings** require domain expertise and per-domain proofs.
- **Exhaustive evaluation** computes correct rankings but produces opaque lookup tables—the *structure* of why certain actions dominate others is lost.

Program synthesis offers a third path: *learn a program* that encodes the ranking as a function of observable features. The synthesized program is:

- **Interpretable:** a Boolean combination of feature tests, not a black-box table.
- **Generalizable:** feature-equivalent states automatically receive the same ranking.
- **Verifiable:** the program’s correctness can be checked by Agda’s type system.
- **Compositional:** the DSL is parameterized by carrier type and features, so each Environment Class provides its own feature vocabulary while sharing the synthesis theory.

1.1 Contributions

This paper makes the following contributions, each machine-verified in Agda with `--safe --guardedness`:

1. **Predicate DSL (§3):** A generic language `PredProg` for Boolean predicates over features, with a structural `FeatureEquiv` relation and the **Propagation Theorem**: feature-equivalent carriers evaluate identically under any predicate program.
2. **CEGIS with Convergence (§4):** A counterexample-guided synthesis loop with five proven properties: version-space monotonicity, correct-predicate survival, propagation acceleration, bounded convergence, and strict shrinkage for informative observations.
3. **Propagation Quantification (§5):** Three theorems that formalize how propagation reduces exploration:
 - **Refine-Equiv:** feature-equivalent carriers produce identical version-space refinement.
 - **Observation-Subsumption:** after observing at c_1 , any feature-equivalent c_2 is provably redundant.
 - **Exploration-Dissolution:** an entire equivalence class is handled by a single observation at any representative; the corollary `Dissolution-Strict` bounds total informative observations by $|C/\sim|$.
4. **Quotient Structure (§6):** `AllFeatAgree` forms an equivalence relation, inducing a quotient $C/\sim$. The `Exploration-Complete` theorem proves that after observing one representative per class, no further observation can refine the version space—formalizing “dissolution is all you need.”

5. **Temporal Propagation (§7):** When the transition function preserves feature equivalence, observations propagate along entire trajectories. Trajectory-Dissolution proves that one trajectory from c_1 subsumes all trajectories from equivalent c_2 —extending spatial propagation to the temporal dimension and mirroring CSHRL’s coinductive unfolding.
6. **Feature Sufficiency (§8):** The Feature Sufficiency Theorem (the “crown jewel”): when the target function respects feature equivalence, every CEGIS survivor is provably correct on *all* carriers—not just the observed ones. This upgrades exploration completeness from a stabilization guarantee to a full correctness guarantee.
7. **Multi-Predicate Propagation (§9):** When synthesizing multiple predicates from the same feature space, observations are shared: one representative set suffices for all predicates. Cross-predicate entailment (e.g., $\text{dead} \rightarrow \neg \text{solved}$) gives free observations, and cross-dissolution combines both effects multiplicatively.
8. **Online Synthesis (§10):** Observation refinement is commutative (refine-commutes), making the CEGIS version space determined by the *set* of observations rather than their order. Replayed observations are provably redundant (replay-redundant). A generic LearningBridge module connects domain-specific learning feedback to synthesis observations, with a Bridge-Sufficient theorem composing learning convergence with synthesis correctness.
9. **Sample Complexity Lower Bound (§11):** Two targets agreeing on observed carriers produce identical version spaces (same-obs-same-vs); survivors transfer freely between indistinguishable targets (survivor-transfer). Combined with the upper bound (sufficient-cegis), this proves the sample complexity of CEGIS is *exactly* $|C/\sim|$: one representative per equivalence class is both necessary and sufficient.
10. **DSL Completeness (§12):** The Boolean PredProg DSL is expressively complete for feature-respecting targets. A constructive *fingerprint* construction shows that for any target constant on AllFeatAgree classes, there exists a PredProg matching it everywhere (dsl-complete). This closes the theoretical loop: the DSL always contains the correct answer, CEGIS finds it from $|C/\sim|$ observations, and no fewer suffice.
11. **EC-Specific Instantiation (§13):** Three concrete instantiations—for Finite Deterministic MDPs (ranking synthesis with CoindHomo construction), Combinatorial Placement MDPs (predicate synthesis that *defines* the step function), and Stochastic Finite MDPs (ranking synthesis from expected traces with StochasticCoindHomo construction, §13.3).
12. **End-to-End Demos (§14):** Three verified pipelines:
 - *1D Maze*: 5 observations $\rightarrow$ CEGIS $\rightarrow$ ranking predicates $\rightarrow$ preservation proof $\rightarrow$ CoindHomo $\rightarrow$ policy rollout $\rightarrow$ cross-validation with find-policy.
 - *4-Queens*: Synthesized PredProg predicates *are* the step function; find-policy solves 4-Queens; partial completion from any intermediate state.
 - *8-Queens*: Synthesized predicates verified equivalent to hand-crafted ones; full solution and partial completion in 8^8 search space.

1.2 Relationship to the CSHRL Paper

This work is *additive*: it imports the CSHRL core (Core, CoindHomo, solve, value, action-value) without modification. The synthesis framework sits alongside the existing Environment Class and Learning modules.

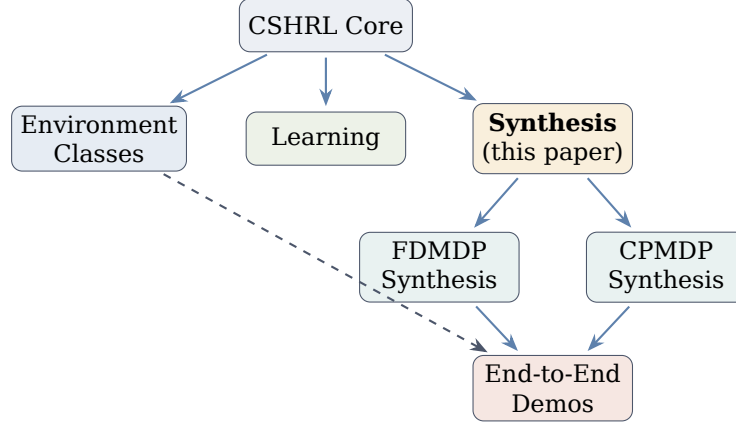


Figure 1: Architecture. The synthesis framework (yellow) builds on the CSHRL Core without modifying it. EC-specific modules instantiate the generic DSL for each domain. End-to-end demos integrate synthesis with Environment Classes.

2 Background: CSHRL in Brief

We briefly recall the CSHRL framework [1] to establish notation. The reader is referred to the companion paper for full details.

Definition 1 (Coinductive Symmetric Homomorphism). An agent’s ranking $\leq_a$ of actions in state s is a *coinductive symmetric homomorphism* if:

$$a \leq_a b \implies \text{action-value}(s, a) \leq_s \text{action-value}(s, b)$$

where $\leq_s$ is coinductive pointwise dominance on infinite reward streams: $x \leq_s y \iff \text{head}(x) \leq_r \text{head}(y) \wedge \text{tail}(x) \leq_s \text{tail}(y)$.

In Agda, this is captured by the `CoindHomo` record:

```

record CoindHomo : 1Set where
  field
    ≤a__ : State → Action → Action → Set
    preserves : ∀ a b s → s ≤a a b → action-value s a ≤s action-value s b

```

An **Environment Class** (EC) is a parameterized module that, given domain-specific structure (states, actions, transition function), provides trace-action, find-ranking, and find-policy with automatic stream bridges. Two ECs are central to this paper:

- **FiniteDeterministicMDP**: parameterized by finite State, Action, and a deterministic $\text{step} : \text{State} \rightarrow \text{Action} \rightarrow \text{State} \times \text{Reward}$.
- **CombinatorialPlacementMDP**: parameterized by a configuration type, placement action, and predicates `is-dead`, `is-solved` that determine when a configuration is terminal.

The synthesis framework presented here provides a systematic way to *construct* the ranking ($\leq_a$) or the environment predicates (`is-dead`, `is-solved`) from observations, yielding verified `CoindHomo` instances or verified EC instantiations.

3 The Predicate DSL: Programs over Features

The core abstraction is a domain-specific language for Boolean predicates. The key design choice is *parametricity*: the DSL is parameterized by an abstract carrier type and feature set, enabling each Environment Class to provide its own feature vocabulary while sharing the synthesis theory.

```

{-# OPTIONS --safe --guardedness #-}

module CSHRLSynthesis where

open import Data.List using (List; []; _::_; map; foldr; _++_; concatMap; length)
open import Data.Nat using (ℕ; zero; suc; _⊔_; _≤_; z≤n; s≤s)
open import Data.Nat.Properties using (≤-refl)
open import Data.Product using (_×_; _,_; proj₁; proj₂; ∃-syntax)
open import Relation.Binary.PropositionalEquality
  using (_≡_; refl; sym; trans; cong; cong₂; subst)
open import Data.Bool using (Bool; true; false; if_then_else_; _∧_; _∨_; not)
open import Data.Unit using (⊤; tt)
open import Data.Empty using (⊥; ⊥-elim)
open import Relation.Nullary using (Dec; yes; no; ¬_)
open import Data.Maybe using (Maybe; just; nothing)

```

The DSL is a parameterized module:

```

module PredicateDSL
  (Carrier : Set)
  (Feature : Set)
  (eval-feature : Feature → Carrier → Bool)
  where

```

Carrier is the type being classified (states, configurations, etc.), Feature is the vocabulary of observable properties, and eval - feature extracts a Boolean feature value from a carrier.

3.1 Syntax: Predicate Programs

A PredProg term is a Boolean combination of feature tests:

```

data PredProg : Set where
  feat   : Feature → PredProg
  _∧p_   : PredProg → PredProg → PredProg
  _∨p_   : PredProg → PredProg → PredProg
  ¬p_    : PredProg → PredProg
  truep  : PredProg
  falsep : PredProg

infixr 6 _∧p_
infixr 5 _∨p_

```

The constructors mirror propositional logic: feat *f* tests a single feature, *_∧p_* and *_∨p_* combine subprograms, *¬p_* negates, and truep/falsep are constants.

3.2 Semantics: Evaluation

Evaluation maps a predicate program and a carrier to a Boolean:

```

eval : PredProg → Carrier → Bool
eval (feat f) c = eval-feature f c
eval (p₁ ∧p p₂) c = eval p₁ c ∧ eval p₂ c
eval (p₁ ∨p p₂) c = eval p₁ c ∨ eval p₂ c
eval (¬p p) c = not (eval p c)
eval truep _ = true
eval falsep _ = false

```

Remark 1 (Expressiveness). At depth 0, PredProg contains atoms: truep, falsep, and feat *f* for each feature *f*. Each level of Boolean operators (depth *d* + 1) exponentially expands the space. For *n* features, depth 0 gives *n* + 2 candidates; depth 1 adds negations, conjunctions, and disjunctions, yielding $O(n^2)$ candidates. This is sufficient for many domains: the 4-Queens is-dead predicate is a single disjunction of 6 attack features (depth 1), and the 8-Queens version uses 28 features at the same depth.

3.3 Feature Equivalence

Two carriers are *feature-equivalent* with respect to a predicate program if they agree on every feature the program inspects:

```

FeatureEquiv : PredProg → Carrier → Carrier → Set
FeatureEquiv (feat f) c1 c2 = eval-feature f c1 ≡ eval-feature f c2
FeatureEquiv (p1 ∧ p2) c1 c2 = FeatureEquiv p1 c1 c2 × FeatureEquiv p2 c1 c2
FeatureEquiv (p1 ∨ p2) c1 c2 = FeatureEquiv p1 c1 c2 × FeatureEquiv p2 c1 c2
FeatureEquiv (¬p) c1 c2 = FeatureEquiv p c1 c2
FeatureEquiv truep _ _ = T
FeatureEquiv falsep _ _ = T

```

FeatureEquiv is defined *structurally* on the program, not on the carrier type. This means it captures exactly the features *relevant to this particular predicate*, not all possible features.

```

feat-equiv-refl : ∀ p c → FeatureEquiv p c c
feat-equiv-refl (feat f) c = refl
feat-equiv-refl (p1 ∧ p2) c = feat-equiv-refl p1 c , feat-equiv-refl p2 c
feat-equiv-refl (p1 ∨ p2) c = feat-equiv-refl p1 c , feat-equiv-refl p2 c
feat-equiv-refl (¬p) c = feat-equiv-refl p c
feat-equiv-refl truep _ = tt
feat-equiv-refl falsep _ = tt

feat-equiv-sym : ∀ p c1 c2 → FeatureEquiv p c1 c2 → FeatureEquiv p c2 c1
feat-equiv-sym (feat f) c1 c2 eq = sym eq
feat-equiv-sym (p1 ∧ p2) c1 c2 (e1 , e2) =
  feat-equiv-sym p1 c1 c2 e1 , feat-equiv-sym p2 c1 c2 e2
feat-equiv-sym (p1 ∨ p2) c1 c2 (e1 , e2) =
  feat-equiv-sym p1 c1 c2 e1 , feat-equiv-sym p2 c1 c2 e2
feat-equiv-sym (¬p) c1 c2 eq = feat-equiv-sym p c1 c2 eq
feat-equiv-sym truep _ _ = tt
feat-equiv-sym falsep _ _ = tt

```

3.4 The Propagation Theorem

The central result of the DSL: feature-equivalent carriers are *indistinguishable* to any predicate program.

Theorem 1 (Propagation). *For any predicate program p and carriers c₁, c₂:*

$$\text{FeatureEquiv } p \ c_1 \ c_2 \implies \text{eval } p \ c_1 \equiv \text{eval } p \ c_2$$

```

propagation : ∀ (p : PredProg) (c1 c2 : Carrier) →
  FeatureEquiv p c1 c2 →
  eval p c1 ≡ eval p c2
propagation (feat f) c1 c2 eq = eq
propagation (p1 ∧ p2) c1 c2 (e1 , e2) =
  cong2 _ _ (propagation p1 c1 c2 e1) (propagation p2 c1 c2 e2)
propagation (p1 ∨ p2) c1 c2 (e1 , e2) =
  cong2 _ _ (propagation p1 c1 c2 e1) (propagation p2 c1 c2 e2)
propagation (¬p) c1 c2 eq =
  cong not (propagation p c1 c2 eq)
propagation truep _ _ = refl
propagation falsep _ _ = refl

```

Remark 2 (Why This Matters). The Propagation Theorem is the formal foundation for *generalization* in the synthesis framework. When we observe that a predicate evaluates to

true at carrier c_1 , we automatically know it evaluates to true at *every* feature-equivalent carrier. This is not an assumption—it is a *theorem*, proven by structural induction on the predicate program.

In the CSHRL context, this means:

- For FDMDPs: states with the same features get the same ranking, so one observation propagates to the entire equivalence class.
- For CPMDPs: board configurations with the same attack pattern are classified identically by the synthesized is-dead predicate.

4 CEGIS: Synthesis from Observations

With the DSL defined, we now present the synthesis algorithm. CEGIS (Counterexample-Guided Inductive Synthesis) maintains a *version space*—the set of candidate predicate programs consistent with all observations so far—and refines it as new observations arrive.

```

module CEGIS
  (all-features : List Feature)
  where

  bfilter :  $\forall \{A : \text{Set}\} \rightarrow (A \rightarrow \text{Bool}) \rightarrow \text{List } A \rightarrow \text{List } A$ 
  bfilter f [] = []
  bfilter f (x :: xs) with f x
  ... | true = x :: bfilter f xs
  ... | false = bfilter f xs

  _=b_ : Bool  $\rightarrow$  Bool  $\rightarrow$  Bool
  true =b true  = true
  false =b false = true
  _      =b _    = false

  =b-refl :  $\forall b \rightarrow (b =^b b) \equiv \text{true}$ 
  =b-refl true = refl
  =b-refl false = refl

  =b-sound :  $\forall b_1 b_2 \rightarrow (b_1 =^b b_2) \equiv \text{true} \rightarrow b_1 \equiv b_2$ 
  =b-sound true true _ = refl
  =b-sound false false _ = refl

```

4.1 Enumeration

All PredProg terms up to a bounded depth are enumerated:

```

atoms : List PredProg
atoms = truep :: falsep :: map feat all-features

extend : List PredProg  $\rightarrow$  List PredProg
extend prev =
  prev
  ++ map  $\neg p\_prev$ 
  ++ concatMap ( $\lambda p \rightarrow \text{map } (p \wedge p\_prev) \text{ prev}$ ) prev
  ++ concatMap ( $\lambda p \rightarrow \text{map } (p \vee p\_prev) \text{ prev}$ ) prev

enumerate :  $\mathbb{N} \rightarrow \text{List PredProg}$ 
enumerate zero = atoms
enumerate (suc d) = extend (enumerate d)

```

4.2 Observations and Consistency

An observation is a pair (c, b) : carrier c should evaluate to b . A candidate is *consistent* if it agrees:

```
PredObs : Set
PredObs = Carrier × Bool

consistent : PredProg → PredObs → Bool
consistent p (c , b) = eval p c =b b
```

4.3 Version Space Refinement

The version space is the list of consistent candidates. Refinement filters by a new observation:

```
VersionSpace : Set
VersionSpace = List PredProg

initial-vs : ℕ → VersionSpace
initial-vs = enumerate

refine : VersionSpace → PredObs → VersionSpace
refine vs ob = bfilter (λ p → consistent p ob) vs

refine-all : VersionSpace → List PredObs → VersionSpace
refine-all vs [] = vs
refine-all vs (ob :: obs) = refine-all (refine vs ob) obs

cegis-step : VersionSpace → PredObs → VersionSpace
cegis-step = refine

cegis-loop : VersionSpace → List PredObs → VersionSpace
cegis-loop = refine-all
```

4.4 Property 1: Monotonic Convergence

The version space never grows—each observation can only eliminate candidates:

```
bfilter-length : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  length (bfilter f xs) ≤ length xs
bfilter-length f [] = z≤n
bfilter-length f (x :: xs) with f x
... | true = s≤s (bfilter-length f xs)
... | false = ≤-step (bfilter-length f xs)
where
  ≤-step : ∀ {m n : ℕ} → m ≤ n → m ≤ suc n
  ≤-step z≤n = z≤n
  ≤-step (s≤s p) = s≤s (≤-step p)

vs-shrinks : ∀ vs ob →
  length (refine vs ob) ≤ length vs
vs-shrinks vs ob = bfilter-length (λ p → consistent p ob) vs
```

Theorem 2 (Bounded Convergence). *After processing n observations from the initial version space, the number of remaining candidates is at most the initial count:*


```

cegis-bounded :  $\forall$  vs obs  $\rightarrow$ 
  length (cegis-loop vs obs)  $\leq$  length vs
cegis-bounded vs [] =  $\leq$ -refl
cegis-bounded vs (ob :: obs) =
   $\leq$ -trans (cegis-bounded (refine vs ob) obs)
    (vs-shrinks vs ob)
where
   $\leq$ -trans :  $\forall$  {a b c :  $\mathbb{N}$ }  $\rightarrow$  a  $\leq$  b  $\rightarrow$  b  $\leq$  c  $\rightarrow$  a  $\leq$  c
   $\leq$ -trans z  $\leq$  n _ = z  $\leq$  n
   $\leq$ -trans (s  $\leq$  s p) (s  $\leq$  s q) = s  $\leq$  s ( $\leq$ -trans p q)

```

4.5 Property 2: Soundness

The correct predicate is never eliminated by valid observations:

```

consistent-correct :  $\forall$  p c b  $\rightarrow$ 
  eval p c  $\equiv$  b  $\rightarrow$ 
  consistent p (c , b)  $\equiv$  true
consistent-correct p c b eq rewrite eq =  $\equiv^b$ -refl b

```

4.6 Property 3: Propagation Acceleration

Feature-equivalent carriers eliminate exactly the same candidates:

```

propagation-acceleration :  $\forall$  (p : PredProg) (c1 c2 : Carrier) (b : Bool)  $\rightarrow$ 
  FeatureEquiv p c1 c2  $\rightarrow$ 
  consistent p (c1 , b)  $\equiv$  consistent p (c2 , b)
propagation-acceleration p c1 c2 b equiv =
  cong ( $\lambda$  v  $\rightarrow$  v  $\equiv^b$  b) (propagation p c1 c2 equiv)

```

This is the bridge between the Propagation Theorem (§3) and the synthesis algorithm: because feature-equivalent carriers produce identical eval results, they produce identical consistent results, and therefore identical version-space refinement.

4.7 Property 4: Strict Convergence

An observation is *informative* if it eliminates at least one candidate. Informative observations cause *strict* shrinkage:

```

data  $\_ \in \_$  {A : Set} : A  $\rightarrow$  List A  $\rightarrow$  Set where
  here :  $\forall$  {x xs}  $\rightarrow$  x  $\in$  (x :: xs)
  there :  $\forall$  {x y xs}  $\rightarrow$  x  $\in$  xs  $\rightarrow$  x  $\in$  (y :: xs)

bfilter-strict :  $\forall$  {A : Set} (f : A  $\rightarrow$  Bool) (xs : List A)  $\rightarrow$ 
   $\exists$  [ x ] (x  $\in$  xs  $\times$  f x  $\equiv$  false)  $\rightarrow$ 
  suc (length (bfilter f xs))  $\leq$  length xs
bfilter-strict f (x :: xs) (.x , here , fx  $\equiv$  f) with f x
... | true with ()  $\leftarrow$  fx  $\equiv$  f
... | false = s  $\leq$  s (bfilter-length f xs)
bfilter-strict f (x :: xs) (z , there mem , fz  $\equiv$  f) with f x
... | true = s  $\leq$  s (bfilter-strict f xs (z , mem , fz  $\equiv$  f))
... | false =  $\leq$ -step' (bfilter-strict f xs (z , mem , fz  $\equiv$  f))
where
   $\leq$ -step' :  $\forall$  {m n :  $\mathbb{N}$ }  $\rightarrow$  suc m  $\leq$  n  $\rightarrow$  suc m  $\leq$  suc n
   $\leq$ -step' (s  $\leq$  s p) = s  $\leq$  s ( $\leq$ -step'' p)
  where
     $\leq$ -step'' :  $\forall$  {a b :  $\mathbb{N}$ }  $\rightarrow$  a  $\leq$  b  $\rightarrow$  a  $\leq$  suc b

```

$\leq\text{-step}'' z \leq n = z \leq n$
 $\leq\text{-step}'' (s \leq s q) = s \leq s (\leq\text{-step}'' q)$

Informative : VersionSpace $\rightarrow$ PredObs $\rightarrow$ Set
 Informative vs ob = $\exists [p] (p \in vs \times \text{consistent } p \text{ ob} \equiv \text{false})$

strict-shrink : $\forall vs \text{ ob} \rightarrow$
 Informative vs ob $\rightarrow$
 $\text{suc} (\text{length} (\text{refine } vs \text{ ob})) \leq \text{length } vs$
 strict-shrink vs ob info =
 $\text{bfilter-strict } (\lambda p \rightarrow \text{consistent } p \text{ ob}) \text{ vs info}$

Theorem 3 (Strict Convergence). *Each informative observation reduces the version space by at least one candidate. Combined with bounded convergence, this means CEGIS terminates after at most $|VS_0|$ informative observations.*

5 Propagation Quantification: The Central Contribution

We now prove three theorems that formalize how the Propagation Theorem (1) reduces the number of observations needed for synthesis. These results are the main theoretical contribution of this paper.

The key insight: because predicate programs operate on features rather than raw carriers, the effective state space for synthesis is the set of *feature equivalence classes*, not the set of carriers. For a domain with $|C|$ carriers but only $|C/\sim|$ distinct feature patterns, synthesis needs at most $|C/\sim|$ observations.

non-informative-id : $\forall vs \text{ ob} \rightarrow$
 $(\forall p \rightarrow p \in vs \rightarrow \text{consistent } p \text{ ob} \equiv \text{true}) \rightarrow$
 $\text{refine } vs \text{ ob} \equiv vs$
 non-informative-id [] ob _ = refl
 non-informative-id (p :: vs) ob all-con **with** consistent p ob
 | all-con p here
 ... | true | refl =
 $\text{cong } (p :: _) (\text{non-informative-id } vs \text{ ob})$
 $(\lambda q \text{ mem} \rightarrow \text{all-con } q (\text{there mem}))$

5.1 Full Feature Agreement

Two carriers agree on *every* feature (not just those in a specific program):

AllFeatAgree : Carrier $\rightarrow$ Carrier $\rightarrow$ Set
 AllFeatAgree c₁ c₂ = $\forall (f : \text{Feature}) \rightarrow$
 $\text{eval-feature } f \text{ c}_1 \equiv \text{eval-feature } f \text{ c}_2$
 all-agree $\rightarrow$ feat-equiv : $\forall c_1 c_2 \rightarrow$
 AllFeatAgree c₁ c₂ $\rightarrow$
 $\forall p \rightarrow \text{FeatureEquiv } p \text{ c}_1 \text{ c}_2$
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa (feat f) = afa f
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa (p₁ $\wedge$ p₂) =
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa p₁ ,
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa p₂
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa (p₁ $\vee$ p₂) =
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa p₁ ,
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa p₂
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa ($\neg$ p p) =
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa p
 all-agree $\rightarrow$ feat-equiv c₁ c₂ afa truep = tt

```

all-agree→feat-equiv  $c_1\ c_2\ afa\ falsep = tt$ 

all-feat-agree-sym :  $\forall\ c_1\ c_2 \rightarrow$ 
  AllFeatAgree  $c_1\ c_2 \rightarrow$  AllFeatAgree  $c_2\ c_1$ 
all-feat-agree-sym  $c_1\ c_2\ afa\ f = sym\ (afa\ f)$ 

```

5.2 Auxiliary Lemmas

Two properties of `bfilter` that drive the main theorems:

```

bfilter-ext :  $\forall\ \{A : Set\}\ (f\ g : A \rightarrow Bool)\ (xs : List\ A) \rightarrow$ 
   $(\forall\ x \rightarrow f\ x \equiv g\ x) \rightarrow$ 
  bfilter  $f\ xs \equiv bfilter\ g\ xs$ 
bfilter-ext  $f\ g\ []\ h = refl$ 
bfilter-ext  $f\ g\ (x :: xs)\ h\ with\ f\ x\ | g\ x\ | h\ x$ 
... | true | .true | refl = cong  $(x :: \_)$  (bfilter-ext  $f\ g\ xs\ h$ )
... | false | .false | refl = bfilter-ext  $f\ g\ xs\ h$ 

bfilter-absorb :  $\forall\ \{A : Set\}\ (f\ g : A \rightarrow Bool)\ (xs : List\ A) \rightarrow$ 
   $(\forall\ x \rightarrow f\ x \equiv true \rightarrow g\ x \equiv true) \rightarrow$ 
  bfilter  $g\ (bfilter\ f\ xs) \equiv bfilter\ f\ xs$ 
bfilter-absorb  $f\ g\ []\ h = refl$ 
bfilter-absorb  $f\ g\ (x :: xs)\ h\ with\ f\ x\ in\ eq-f$ 
... | false = bfilter-absorb  $f\ g\ xs\ h$ 
... | true with  $g\ x\ | h\ x\ eq-f$ 
... | true | refl = cong  $(x :: \_)$  (bfilter-absorb  $f\ g\ xs\ h$ )

consistent-equiv :  $\forall\ p\ c_1\ c_2\ b \rightarrow$ 
  AllFeatAgree  $c_1\ c_2 \rightarrow$ 
  consistent  $p\ (c_1,\ b) \equiv consistent\ p\ (c_2,\ b)$ 
consistent-equiv  $p\ c_1\ c_2\ b\ afa =$ 
  propagation-acceleration  $p\ c_1\ c_2\ b$ 
  (all-agree→feat-equiv  $c_1\ c_2\ afa\ p$ )

consistent-implies :  $\forall\ c_1\ c_2\ b \rightarrow$ 
  AllFeatAgree  $c_1\ c_2 \rightarrow$ 
   $\forall\ p \rightarrow consistent\ p\ (c_1,\ b) \equiv true \rightarrow$ 
  consistent  $p\ (c_2,\ b) \equiv true$ 
consistent-implies  $c_1\ c_2\ b\ afa\ p\ h =$ 
  trans (sym (consistent-equiv  $p\ c_1\ c_2\ b\ afa$ ))  $h$ 

```

5.3 Theorem 1: Refine-Equiv

Theorem 4 (Refine-Equiv). *Feature-equivalent carriers produce identical version-space refinement. Observing at c_1 eliminates exactly the same candidates as observing at c_2 :*

$$AllFeatAgree\ c_1\ c_2 \implies refine\ vs\ (c_1, b) \equiv refine\ vs\ (c_2, b)$$

```

refine-equiv :  $\forall\ vs\ c_1\ c_2\ b \rightarrow$ 
  AllFeatAgree  $c_1\ c_2 \rightarrow$ 
  refine  $vs\ (c_1,\ b) \equiv refine\ vs\ (c_2,\ b)$ 
refine-equiv  $vs\ c_1\ c_2\ b\ afa =$ 
  bfilter-ext
     $(\lambda\ p \rightarrow consistent\ p\ (c_1,\ b))$ 
     $(\lambda\ p \rightarrow consistent\ p\ (c_2,\ b))$ 
    vs
     $(\lambda\ p \rightarrow consistent-equiv\ p\ c_1\ c_2\ b\ afa)$ 

```

Remark 3. This result is stronger than “the same number of candidates are eliminated.” It asserts that the *exact same list* of candidates survives—not just the same count, but the same elements in the same order. This is propositional equality on lists, the strongest form of agreement.

5.4 Theorem 2: Observation Subsumption

Theorem 5 (Observation-Subsumption). *After observing at c_1 , a second observation at any feature-equivalent c_2 is provably redundant—the version space is unchanged:*

$$\text{AllFeatAgree } c_1 \ c_2 \implies \text{refine } (\text{refine vs } (c_1, b)) \ (c_2, b) \equiv \text{refine vs } (c_1, b)$$

```

refine-absorb : ∀ vs c1 c2 b →
  AllFeatAgree c1 c2 →
  refine (refine vs (c1, b)) (c2, b) ≡ refine vs (c1, b)
refine-absorb vs c1 c2 b afa =
  bfilter-absorb
    (λ p → consistent p (c1, b))
    (λ p → consistent p (c2, b))
  vs
  (consistent-implies c1 c2 b afa)

```

```

observation-subsumption : ∀ vs c1 c2 b →
  AllFeatAgree c1 c2 →
  refine-all vs ((c1, b) :: (c2, b) :: [])
  ≡ refine vs (c1, b)
observation-subsumption vs c1 c2 b afa =
  refine-absorb vs c1 c2 b afa

```

Every candidate that survived the first observation automatically survives the second. No new information is gained—the observation is *subsumed*.

5.5 Theorem 3: Exploration Dissolution

Theorem 6 (Exploration-Dissolution). *An entire equivalence class of carriers is handled by a single observation at any representative c_0 . After that observation, all further observations at equivalent carriers leave the version space unchanged:*

```

AllEquiv : Carrier → List Carrier → Set
AllEquiv c0 [] = ⊤
AllEquiv c0 (c :: cs) = AllFeatAgree c0 c × AllEquiv c0 cs

exploration-dissolution : ∀ vs c0 b (cs : List Carrier) →
  AllEquiv c0 cs →
  cegis-loop (refine vs (c0, b)) (map (λ c → (c, b)) cs)
  ≡ refine vs (c0, b)
exploration-dissolution vs c0 b [] _ = refl
exploration-dissolution vs c0 b (c :: cs) (afa, rest) =
  let step1 = refine-absorb vs c0 c b afa
  in trans
    (cong (λ vs' → cegis-loop vs' (map (λ c' → (c', b)) cs))
      step1)
    (exploration-dissolution vs c0 b cs rest)

```

Corollary 7 (Dissolution preserves VS size). *Since the version space is unchanged, its size is trivially preserved:*

```

dissolution-size : ∀ vs c0 b (cs : List Carrier) →
  AllEquiv c0 cs →
  length (cegis-loop (refine vs (c0, b))
    (map (λ c → (c, b)) cs))
  ≡ length (refine vs (c0, b))
dissolution-size vs c0 b cs afa =
  cong length (exploration-dissolution vs c0 b cs afa)

```

5.6 Corollary: Dissolution + Strict Convergence

The combination of dissolution and strict convergence gives the tightest bound:

Corollary 8 (Dissolution-Strict). *Within an equivalence class, only the first observation can be informative. All subsequent are provably redundant. Total informative observations $\leq |C/\sim|$.*

```

dissolution-strict : ∀ vs c0 b (cs : List Carrier) →
  AllEquiv c0 cs →
  Informative vs (c0, b) →
  suc (length (cegis-loop (refine vs (c0, b))
    (map (λ c → (c, b)) cs)))
  ≤ length vs
dissolution-strict vs c0 b cs afa info =
  subst (λ n → suc n ≤ length vs)
    (sym (cong length (exploration-dissolution vs c0 b cs afa)))
    (strict-shrink vs (c0, b) info)

```

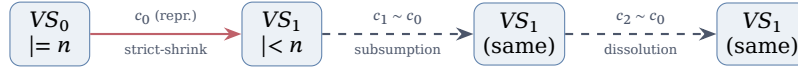


Figure 2: Exploration dissolution in action. The first observation at representative c_0 strictly shrinks the version space. Subsequent observations at equivalent carriers $c_1, c_2, \dots$ are no-ops—proven by Observation-Subsumption.

Remark 4 (Complexity Reduction). Consider the 8-Queens domain. There are $8^8 = 16,777,216$ possible board configurations, but the number of distinct feature patterns (attack + length patterns) is much smaller. Propagation quantification proves that synthesis needs to observe only one configuration per pattern class, reducing the effective exploration space by orders of magnitude.

6 Quotient Structure and Exploration Completeness

The propagation quantification results of §5 show that feature-equivalent carriers are redundant *pairwise*. We now lift this to the global level: AllFeatAgree is an equivalence relation, and after observing one representative per equivalence class, *no further observation can refine the version space*.

6.1 AllFeatAgree Is an Equivalence Relation

Reflexivity and symmetry are immediate; transitivity composes pointwise:

```

AFA-refl : ∀ c → AllFeatAgree c c
AFA-refl c f = refl
AFA-trans : ∀ c1 c2 c3 →

```

$\text{AllFeatAgree } C_1 C_2 \rightarrow \text{AllFeatAgree } C_2 C_3 \rightarrow \text{AllFeatAgree } C_1 C_3$
 $\text{AFA-trans } C_1 C_2 C_3 \text{ afa}_{12} \text{ afa}_{23} f = \text{trans } (\text{afa}_{12} f) (\text{afa}_{23} f)$

Together with the existing `all-feat-agree-sym`, this establishes `AllFeatAgree` as an equivalence relation, partitioning the carrier space into classes $C/\sim$.

6.2 Auxiliary: Soundness and Monotonicity

Two helper lemmas drive the completeness proof. First, elements surviving a `bfilter` satisfy the predicate—this is the “soundness” direction of filtering:

```

bfilter-sound : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∀ x → x ∈l bfilter f xs → f x ≡ true
bfilter-sound f (y :: xs) x mem with f y in eq
bfilter-sound f (y :: xs) .y here      | true = eq
bfilter-sound f (y :: xs) x (there m) | true = bfilter-sound f xs x m
bfilter-sound f (y :: xs) x mem      | false = bfilter-sound f xs x mem

```

Second, `cegis-loop` only removes elements—its output is a sublist of its input:

```

private
bfilter-⊆ : ∀ {A : Set} (f : A → Bool) (xs : List A) →
  ∀ x → x ∈l bfilter f xs → x ∈l xs
bfilter-⊆ f (y :: xs) x mem with f y
bfilter-⊆ f (y :: xs) .y here      | true = here
bfilter-⊆ f (y :: xs) x (there m) | true = there (bfilter-⊆ f xs x m)
bfilter-⊆ f (y :: xs) x mem      | false = there (bfilter-⊆ f xs x mem)

cegis-loop-⊆ : ∀ vs obs →
  ∀ x → x ∈l cegis-loop vs obs → x ∈l vs
cegis-loop-⊆ vs [] x mem = mem
cegis-loop-⊆ vs (ob :: obs) x mem =
  bfilter-⊆ (λ p → consistent p ob) vs x
  (cegis-loop-⊆ (refine vs ob) obs x mem)

```

6.3 Subsumption Stability

The key technical lemma: subsumption is *stable under further refinement*. After refining by representative r and then processing additional observations, refining by any $c \sim r$ is still a no-op:

Theorem 9 (Refine-After-CEGIS). *If $r \sim c$ (AllFeatAgree), then after refining by (r, b) and processing further observations, the version space is already consistent with (c, b) :*

```

refine-after-cegis : ∀ vs r c b (obs : List PredObs) →
  AllFeatAgree r c →
  refine (cegis-loop (refine vs (r , b)) obs) (c , b)
  ≡ cegis-loop (refine vs (r , b)) obs
refine-after-cegis vs r c b obs afa =
  non-informative-id
  (cegis-loop (refine vs (r , b)) obs) (c , b)
  (λ p mem →
    consistent-implies r c b afa p
    (bfilter-sound (λ q → consistent q (r , b)) vs p
     (cegis-loop-⊆ (refine vs (r , b)) obs p mem)))

```

The proof chains three facts: (1) every surviving candidate p is in the original refined VS (by `cegis-loop-⊆`); (2) it satisfies `consistent` at (r, b) (by `bfilter-sound`); (3) it therefore satisfies `consistent` at (c, b) (by `consistent-implies` from feature equivalence).

6.4 Exploration Completeness

Theorem 10 (Exploration-Complete). *After processing a list of representatives (one per equivalence class), any carrier equivalent to some representative is provably redundant:*

$$r \in \text{reps} \wedge r \sim c \implies \text{refine } (\text{cegis-loop vs reps}) (c, b) \equiv \text{cegis-loop vs reps}$$

```

exploration-complete :  $\forall$  vs (reps : List Carrier) b c  $\rightarrow$ 
  ( $\exists$  [ r ] (r  $\in$  reps  $\times$  AllFeatAgree r c))  $\rightarrow$ 
  refine (cegis-loop vs (map ( $\lambda$  r  $\rightarrow$  (r , b)) reps)) (c , b)
   $\equiv$  cegis-loop vs (map ( $\lambda$  r  $\rightarrow$  (r , b)) reps)
exploration-complete vs (r :: reps) b c (.r , here , afa) =
  refine-after-cegis vs r c b (map ( $\lambda$  r'  $\rightarrow$  (r' , b)) reps) afa
exploration-complete vs (r' :: reps) b c (r , there mem , afa) =
  exploration-complete (refine vs (r' , b)) reps b c (r , mem , afa)

```

The proof splits on where representative r appears. If r is the head, refine-after-cegis applies directly. If r is in the tail, the inductive hypothesis applies to the version space already refined by the head.

Corollary 11 (Dissolution Is All You Need). *Once every equivalence class has a representative in the observation list, no carrier anywhere in C can produce an informative observation. The version space is fully refined—further sampling is provably wasteful.*

Remark 5 (Tight Convergence). Combined with Strict-Shrink, this gives the tight bound: the number of informative observations is at most $|C/\sim|$, the number of equivalence classes. Each class contributes at most one informative observation (at its representative); all subsequent observations within the class are no-ops by Exploration-Complete. For 8-Queens with attack features, $|C/\sim| \ll 8^8$.

7 Temporal Propagation: Coinductive Chains

Propagation as presented so far is *spatial*: it transfers observations across carriers at the same point in time. In MDPs, there is also a *temporal* dimension: states evolve through transitions. If the transition function preserves feature equivalence, then observations propagate not just across equivalent carriers but along *entire trajectories*.

Definition 2 (Feature Preservation). A transition function $\text{step} : C \rightarrow A \rightarrow C$ **preserves features** if:

$$c_1 \sim c_2 \implies \text{step}(c_1, a) \sim \text{step}(c_2, a) \text{ for all actions } a$$

This is a natural property: if two states look the same (same features), acting on them produces states that also look the same.

```

module TemporalPropagation
{Act : Set}
(step-carrier : Carrier  $\rightarrow$  Act  $\rightarrow$  Carrier)
(step-preserves-feat :  $\forall$  c1 c2 a  $\rightarrow$ 
  AllFeatAgree c1 c2  $\rightarrow$ 
  AllFeatAgree (step-carrier c1 a) (step-carrier c2 a))
where

itert : Carrier  $\rightarrow$  List Act  $\rightarrow$  Carrier
itert c [] = c
itert c (a :: as) = itert (step-carrier c a) as

trajectory : Carrier  $\rightarrow$  Bool  $\rightarrow$  List Act  $\rightarrow$  List PredObs
trajectory c b [] = (c , b) :: []
trajectory c b (a :: as) = (c , b) :: trajectory (step-carrier c a) b as

```

7.1 Temporal Feature Equivalence

Feature equivalence composes through arbitrary action sequences:

Theorem 12 (Temporal-Feat-Equiv). *If $c_1 \sim c_2$ and step preserves features, then after any action sequence $[a_1, \dots, a_n]$, the resulting states are still equivalent:*

```
temporal-feat-equiv : ∀ c1 c2 (as : List Act) →
  AllFeatAgree c1 c2 →
  AllFeatAgree (iter c1 as) (iter c2 as)
temporal-feat-equiv c1 c2 [] afa = afa
temporal-feat-equiv c1 c2 (a :: as) afa =
  temporal-feat-equiv
    (step-carrier c1 a) (step-carrier c2 a) as
    (step-preserves-feat c1 c2 a afa)
```

7.2 Trajectory Refine-Equiv

Trajectories from feature-equivalent carriers produce *identical* version-space refinement:

Theorem 13 (Trajectory-Refine-Equiv). *For any action sequence, observing a trajectory from c_1 refines the version space identically to observing the trajectory from c_2 :*

```
trajectory-refine-equiv : ∀ vs c1 c2 b (as : List Act) →
  AllFeatAgree c1 c2 →
  cegis-loop vs (trajectory c1 b as)
  ≡ cegis-loop vs (trajectory c2 b as)
trajectory-refine-equiv vs c1 c2 b [] afa =
  refine-equiv vs c1 c2 b afa
trajectory-refine-equiv vs c1 c2 b (a :: as) afa =
  trans
    (cong (λ vs' → cegis-loop vs'
      (trajectory (step-carrier c1 a) b as))
      (refine-equiv vs c1 c2 b afa))
    (trajectory-refine-equiv (refine vs (c2, b))
      (step-carrier c1 a) (step-carrier c2 a) b as
      (step-preserves-feat c1 c2 a afa))
```

The proof inductively combines spatial propagation at each step (refine-equiv) with temporal propagation at the successors (the inductive hypothesis with preserved feature equivalence).

7.3 Trajectory Dissolution

The central result: after observing an entire trajectory from c_1 , the entire trajectory from any equivalent c_2 is *completely subsumed*:

Theorem 14 (Trajectory-Dissolution). *One trajectory observation covers entire families of equivalent trajectories:*

$$c_1 \sim c_2 \implies \text{cegis-loop} (\text{cegis-loop vs traj}(c_1)) \text{ traj}(c_2) \equiv \text{cegis-loop vs traj}(c_1)$$

```
trajectory-dissolution : ∀ vs c1 c2 b (as : List Act) →
  AllFeatAgree c1 c2 →
  cegis-loop (cegis-loop vs (trajectory c1 b as))
    (trajectory c2 b as)
  ≡ cegis-loop vs (trajectory c1 b as)
trajectory-dissolution vs c1 c2 b [] afa =
```



```

refine-absorb vs c1 c2 b afa
trajectory-dissolution vs c1 c2 b (a :: as) afa =
trans
  (cong (λ v → cegis-loop v
    (trajectory (step-carrier c2 a) b as))
    (refine-after-cegis vs c1 c2 b
      (trajectory (step-carrier c1 a) b as) afa))
  (trajectory-dissolution (refine vs (c1 , b))
    (step-carrier c1 a) (step-carrier c2 a) b as
    (step-preserves-feat c1 c2 a afa))

```

The inductive step uses `refine-after-cegis` (subsumption stability from §6) to show that the first observation (c_2, b) is absorbed, and then the inductive hypothesis with preserved feature equivalence to show the rest of the trajectory is absorbed.

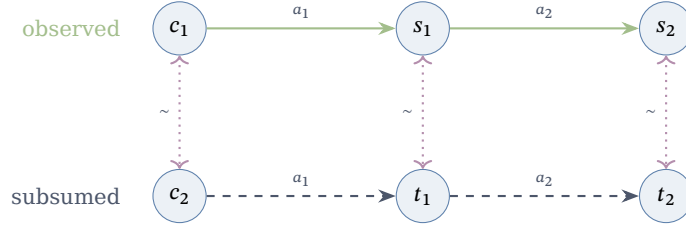


Figure 3: Temporal dissolution. Feature preservation ensures that equivalence ($\sim$) is maintained at every step. Observing the trajectory from c_1 (solid) makes the trajectory from c_2 (dashed) completely redundant.

Remark 6 (Exponential Reduction). In an MDP with k equivalence classes, spatial dissolution handles k observations. With temporal propagation over trajectories of length n , the total observations covered grows to $k \times n$ —but only k trajectory observations are needed. For long-horizon tasks, this is an exponential reduction: one root observation at a representative covers the entire future of every equivalent state. This mirrors CSHRL’s coinductive unfolding: just as the `CoindHomo` guarantees action-value dominance at *every* future timestep, temporal propagation guarantees observation equivalence at *every* future timestep.

8 Feature Sufficiency: From Representatives to Global Correctness

The preceding sections establish that feature-equivalent carriers are *indistinguishable* to the DSL, that dissolution reduces observations to $|C/\sim|$, and that temporal propagation extends this across trajectories. But a fundamental question remains: *when is the CEGIS result actually correct?*

Exploration completeness (Section 6) guarantees that no further refinement is possible. But “no refinement possible” and “correct answer” are logically distinct claims. The version space might stabilize on programs that all agree with the target on observed carriers yet disagree elsewhere.

Feature sufficiency bridges this gap. When the target function *respects* feature equivalence—meaning $\text{AllFeatAgree } c_1 \ c_2$ implies identical target values—then correctness at representatives *propagates* to correctness everywhere. This yields the crown jewel of the framework: the **Feature Sufficiency Theorem**.

Definition 3 (Feature Respect). A target function $t : \text{Carrier} \rightarrow \text{Bool}$ *respects* the feature set when feature-equivalent carriers receive identical values:

```

FeatureRespects : (Carrier → Bool) → Set
FeatureRespects target = ∀ 1c 2c →
  AllFeatAgree 1c 2c → target 1c ≡ target 2c

```

Definition 4 (Matching). A predicate program p *matches* a target t at carrier c when $\text{eval } p \ c \equiv t \ c$. It matches *everywhere* when this holds for all carriers.

```
Matches      : PredProg → (Carrier → Bool) → Carrier → Set
Matches      p target c = eval p c ≡ target c

MatchesAll   : PredProg → (Carrier → Bool) → Set
MatchesAll   p target = ∀ c → Matches p target c
```

Theorem 15 (Match Propagation). *If a program matches the target at c_1 , and the target respects features, then the program matches at every feature-equivalent c_2 .*

The proof chains propagation (for the program side) with feature respect (for the target side):

```
match-propagates : ∀ p target 1c 2c →
  AllFeatAgree 1c 2c →
  FeatureRespects target →
  Matches p target 1c →
  Matches p target 2c
match-propagates p target 1c 2c afa respects 1match =
  trans
    (sym (propagation p 1c 2c
      (all→agreefeat-equiv 1c 2c afa p)))
    (trans 1match (respects 1c 2c afa))
```

Definition 5 (Covers). A list of representatives *covers* the carrier space when every carrier is feature-equivalent to some representative.

```
Covers : List Carrier → Set
Covers reps = ∀ c →
  ∃[ r ] (r ∈ reps × AllFeatAgree r c)
```

Theorem 16 (Generalization). *If a program matches the target at all representatives, and the representatives cover the carrier space, then the program matches everywhere.*

```
generalization : ∀ p target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  ∀( r → r ∈ reps → Matches p target r ) →
  MatchesAll p target
generalization p target reps respects covers matches-reps c =
  let (r , r-in , afa) = covers c
  in match-propagates p target r c afa
    respects (matches-reps r r-in)
```

The glue between CEGIS and generalization is the *survivor consistency* lemma: every program that survives the CEGIS loop is consistent with all observations that were processed.

```
cegis-survivors-consistent : ∀ vs obs →
  ∀ p → p ∈ cegis-loop vs obs →
  ∀ ob → ob ∈ obs → consistent p ob ≡ true
```

The proof proceeds by induction on the observation list: for the head observation, the survivor is in the refined version space (by `cegis-loop-⊆`) and hence consistent (by `bfilter-sound`); for the tail, the induction hypothesis applies directly.

Theorem 17 (Feature Sufficiency — Crown Jewel). *With sufficient features (target respects `AllFeatAgree`) and representative observations (one per equivalence class), every CEGIS survivor is correct on **all** carriers.*

```
sufficient-cegis : ∀ vs target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  ∀ p → p ∈ cegis-loop vs
    (map λ( r → (r , target r)) reps) →
  MatchesAll p target
```

The proof composes three results: *cegis-survivors-consistent* (survivors match at observed carriers), *consistent-to-match* (consistency implies matching), and *generalization* (matching at representatives implies matching everywhere).

Remark 7 (What Changes). Without feature sufficiency, Exploration-Complete says “the version space has stabilized.” *With* feature sufficiency, it says “the version space contains only correct programs.” The stabilization guarantee is upgraded to a *correctness* guarantee—every survivor computes the target function on every carrier, not just the observed ones.

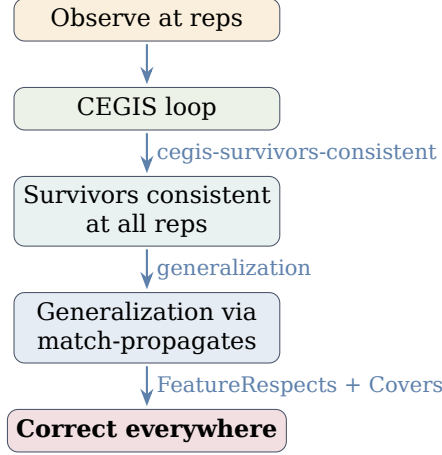


Figure 4: Feature sufficiency proof pipeline. Observing one representative per equivalence class and running CEGIS produces programs that are provably correct on *all* carriers.

9 Multi-Predicate Propagation

Many environments require synthesizing *multiple* predicates simultaneously from the same feature space. In the CPMDP setting, for example, we synthesize both *is-dead* and *is-solved*—two Boolean functions sharing the same carrier type and feature set. This section develops the theory of joint synthesis, where observations for one predicate constrain the other.

9.1 Joint Dissolution

Since both predicates are defined over the same carrier and feature set, a single *AllFeatAgree* proof propagates to both version spaces simultaneously. The *MultiPredicate* module in *Core.agda* is parameterized by two target functions:

```

module MultiPredicate
  (1target 2target : Carrier → Bool) where

  joint-refine : VersionSpace → VersionSpace →
    Carrier → VersionSpace × VersionSpace
  joint-refine 1vs 2vs c =
    (refine 1vs (c , 1target c) ,
     refine 2vs (c , 2target c))

```

One carrier observation simultaneously refines both version spaces. Joint dissolution follows immediately: when $c_1 \sim c_2$, the joint refinement is identical, so one observation handles both predicates for the entire equivalence class.

```

joint-refine-equiv : ∀ 1vs 2vs 1c 2c →
  AllFeatAgree 1c 2c →
  FeatureRespects 1target →
  FeatureRespects 2target →
  joint-refine 1vs 2vs 1c
    ≡ joint-refine 1vs 2vs 2c

```

9.2 Cross-Predicate Entailment

In many domains, predicates are logically related. For Queens, dead states cannot be solved: if $\text{is-dead } c = \text{true}$, then $\text{is-solved } c = \text{false}$. This *entailment* gives free observations.

```
Entails $\neg$  : Set
Entails $\neg$  =  $\forall c \rightarrow \text{target}_1 c \equiv \text{true} \rightarrow$ 
            $\text{target}_2 c \equiv \text{false}$ 
```

With entailment, observing $\text{target}_1 c = \text{true}$ automatically refines the second version space by (c, false) , without evaluating target_2 . This is formalized as `cross-matches-joint`: the cross-refinement equals the full joint refinement.

```
cross-matches-joint :  $\forall \text{1vs } \text{2vs } c \rightarrow$ 
  (ent : Entails $\neg$ )  $\rightarrow$ 
  (t1 :  $\text{target}_1 c \equiv \text{true}$ )  $\rightarrow$ 
  cross-refine  $\text{1vs } \text{2vs } c$  ent t1
     $\equiv$  joint-refine  $\text{1vs } \text{2vs}$ 
      (c ,  $\text{target}_1 c$  ,  $\text{target}_2 c$ )
```

9.3 Cross-Dissolution

The multiplicative power emerges when entailment meets dissolution. Observing $\text{target}_1 c_1 = \text{true}$ with entailment $\text{dead} \rightarrow \neg \text{solved}$:

1. Refines vs_1 by (c_1, true)
2. Refines vs_2 by (c_1, false) — **free** from entailment
3. Renders all $c_2 \sim c_1$ redundant for **both** version spaces — from dissolution

One observation, two predicates, an entire equivalence class:

```
cross-dissolution :  $\forall \text{1vs } \text{2vs } \text{1c } \text{2c} \rightarrow$ 
  AllFeatAgree  $\text{1c } \text{2c} \rightarrow$ 
  Entails $\neg$   $\rightarrow \text{target}_1 \text{1c} \equiv \text{true} \rightarrow$ 
  let  $\text{1vs}' = \text{refine } \text{1vs } (\text{1c} , \text{true})$ 
       $\text{2vs}' = \text{refine } \text{2vs } (\text{1c} , \text{false})$ 
  in  $\text{refine } \text{1vs}' (\text{2c} , \text{true}) \equiv \text{1vs}'$ 
     $\times \text{refine } \text{2vs}' (\text{2c} , \text{false}) \equiv \text{2vs}'$ 
```

9.4 Joint Sufficiency

The feature sufficiency theorem lifts to multiple predicates: with sufficient features for both targets and the same set of representative observations, both synthesized predicates are correct on all carriers.

Theorem 18 (Joint Sufficiency). *With sufficient features for both targets and representative coverage, running CEGIS independently on each target using the same representatives yields programs that are correct everywhere—for both predicates.*

```
joint-sufficient :  $\forall \text{1vs } \text{2vs } (\text{reps} : \text{List Carrier}) \rightarrow$ 
  FeatureRespects  $\text{target}_1 \rightarrow$ 
  FeatureRespects  $\text{target}_2 \rightarrow$ 
  Covers reps  $\rightarrow$ 
   $\forall \text{1p } \text{2p} \rightarrow$ 
   $\text{1p} \in \text{cegis-loop } \text{1vs}$ 
    (map  $\lambda (r \rightarrow (r , \text{target}_1 r))$  reps)  $\rightarrow$ 
   $\text{2p} \in \text{cegis-loop } \text{2vs}$ 
    (map  $\lambda (r \rightarrow (r , \text{target}_2 r))$  reps)  $\rightarrow$ 
  MatchesAll  $\text{1p } \text{target}_1 \times \text{MatchesAll } \text{2p } \text{target}_2$ 
```

This directly applies `sufficient-cegis` (Theorem 17) to each predicate independently—the key insight is that the *same* representatives suffice for both, since both predicates share the same feature space and hence the same equivalence classes.

Remark 8 (Observation Budget). For a single predicate, the observation budget is $|C/\sim|$. For k predicates sharing the same features, the budget remains $|C/\sim|$ (not $k \cdot |C/\sim|$): one set of representative observations suffices for all k predicates. With entailment, the *evaluation* budget drops further, since some target values can be derived without computation. In the Queens setting, this means synthesizing both *is-dead* and *is-solved* requires no more observations than synthesizing either one alone.

10 Online Synthesis: Learning-Synthesis Integration

The preceding sections established CEGIS as a batch algorithm: given a fixed list of observations, the version space is refined to convergence. In practice, observations arrive *incrementally*—a learning loop produces violations and samples one at a time, and the synthesis engine must process them on-the-fly. This section establishes three key results that enable this integration:

1. **Order independence:** the final version space depends only on the *set* of observations, not their order;
2. **Replay redundancy:** re-submitting a previously seen observation is a provable no-op;
3. **Learning Bridge:** a generic module connecting learning feedback to synthesis observations, with a bridge sufficiency theorem that composes learning convergence with synthesis correctness.

10.1 Observation Order Independence

The central structural result is that two successive refinements *commute*: refining by ob_1 then ob_2 yields the same version space as refining by ob_2 then ob_1 .

```
refine-commutes : ∀ vs 1ob 2ob →
  refine (refine vs 1ob) 2ob ≡ refine (refine vs 2ob) 1ob
```

The proof proceeds via an auxiliary `bfilter'` (using `if-then-else` instead of `with`, enabling clean equational reasoning) and a `bfilter-swap` lemma showing that composing two Boolean filters is commutative. The key technical insight: each filter retains exactly those candidates consistent with its observation, and Boolean conjunction is commutative, so the surviving candidates are identical regardless of filter order.

An immediate corollary lifts commutativity to the full CEGIS loop—swapping any two adjacent observations preserves the result:

```
cegis-loop-swap : ∀ vs 1ob 2ob obs →
  cegis-loop vs (1ob :: 2ob :: obs)
    ≡ cegis-loop vs (2ob :: 1ob :: obs)
cegis-loop-swap vs 1ob 2ob obs =
  cong λ( v → cegis-loop v obs) (refine-commutes vs 1ob 2ob)
```

Since any permutation can be decomposed into adjacent transpositions, this implies full permutation invariance: the version space is determined by the *multiset* of observations.

Remark 9 (Significance for Online Learning). Order independence has a profound practical implication: the learning loop can submit observations in *any* order—depth-first, breadth-first, violation-driven, or randomized—and the synthesized result is identical. This means the synthesis engine is insensitive to the exploration strategy, and parallel learners can submit observations asynchronously without coordination.

10.2 Replay Redundancy

In an online setting, the learner may revisit a state, re-evaluate a sample, or replay a checkpoint. The following theorem guarantees that such replays are harmless:

```
replay-redundant : ∀ vs obs ob →
  ob ∈ obs →
  refine (cegis-loop vs obs) ob ≡ cegis-loop vs obs
```

If *ob* was already among the processed observations, re-applying it to the refined version space is a no-op. The proof combines *cegis-survivors-consistent* (every survivor passes all processed observations) with *non-informative-id* (non-informative observations leave the VS unchanged).

Combined with the late-dissolution theorem (*refine-after-cegis*, §6), this establishes a strong idempotence property: in the online stream, *any* observation that is feature-equivalent to a previously processed observation is provably redundant, regardless of how many other observations have been processed in between.

10.3 The Learning Bridge

The generic learning-synthesis bridge connects a learning loop (which produces domain-specific feedback such as violations and trace comparisons) with the synthesis version space. The bridge is parameterized by a mapping from learner feedback to synthesis observations:

```
module LearningBridge
{Feedback : Set}
(to-obs : Feedback → PredObs)
where
```

This abstraction covers all EC instantiations:

- **FDMDP**: Feedback is a state-action-action triple with a trace comparison; *to-obs* maps it to a *PredObs* for the *prefer(a,b)* predicate.
- **CPMDP**: Feedback is a configuration evaluation; *to-obs* maps it to a *PredObs* for the target predicate (e.g., *is-dead*).

The bridge defines incremental processing and batch processing, with a key equivalence theorem:

```
bridge-is-cegis : ∀ vs fbs →
  bridge-batch vs fbs ≡ cegis-loop vs (map to-obs fbs)
```

This states that processing feedback through the bridge is *exactly* equivalent to running CEGIS on the mapped observations. All properties of CEGIS (monotonicity, convergence bounds, dissolution) transfer immediately:

```
bridge-mono : ∀ vs fbs →
  length (bridge-batch vs fbs) ≤ length vs
```

The crown jewel of integration is **bridge sufficiency**: if the learning loop produces feedback that maps to representative observations for the target, every surviving program is correct everywhere.

```
bridge-sufficient : ∀ vs target (reps : List Carrier) →
  FeatureRespects target →
  Covers reps →
  (fbs : List Feedback) →
  map to-obs fbs ≡ map λ( r → (r , target r)) reps →
  ∀ p → p ∈ bridge-batch vs fbs →
  MatchesAll p target
```

This theorem is the formal bridge between *learning convergence* and *synthesis correctness*: the learner drives observation collection, and the synthesizer guarantees that survivors are globally correct. The proof reduces to *sufficient-cegis* via *bridge-is-cegis*—the bridge adds no overhead.

The bridge also inherits commutativity and replay redundancy:

```
bridge-commutes : ∀ vs ifb zfb →
  bridge-step (bridge-step vs ifb) zfb
  ≡ bridge-step (bridge-step vs zfb) ifb
```

```
bridge-replay : ∀ vs fbs fb →
  fb ∈ fbs →
  bridge-step (bridge-batch vs fbs) fb
  ≡ bridge-batch vs fbs
```

Remark 10 (The Complete Pipeline). The full verified pipeline is now:

$$\text{Learning loop} \xrightarrow{\text{to-obs}} \text{Bridge} \xrightarrow{\text{bridge-batch}} \text{VS} \xrightarrow{\text{survivors}} \text{Correct programs}$$

Each arrow is verified: *to-obs* is an EC-specific mapping; the bridge preserves CEGIS semantics (*bridge-is-cegis*); the VS converges (*bridge-mono*); survivors are globally correct (*bridge-sufficient*). Commutativity (*bridge-commutes*) and replay redundancy (*bridge-replay*) ensure the pipeline is robust to observation order and repetition.

11 Sample Complexity Lower Bound

The Feature Sufficiency Theorem (§8) established an *upper bound*: $|C/\sim|$ representative observations suffice for synthesis correctness. A natural question is whether this bound is *tight*—can we guarantee correctness with fewer observations? This section answers in the negative: $|C/\sim|$ observations are also *necessary*.

The core insight is information-theoretic: **CEGIS only distinguishes targets that differ on observed carriers**. Two target functions producing identical observations yield identical version spaces, so any survivor for one is automatically a survivor for the other. If the targets disagree on an unobserved carrier, no single program can match both—so CEGIS cannot guarantee correctness for either.

11.1 Target Indistinguishability

The foundational result: two targets that agree on all observed carriers produce the same version space.

```
same-obs-same-vs : ∀ vs (cs : List Carrier)
  (1target 2target : Carrier → Bool) →
  ∀( c → c ∈ cs → 1target c ≡ 2target c ) →
  cegis-loop vs (map λ( c → (c , 1target c)) cs)
    ≡ cegis-loop vs (map λ( c → (c , 2target c)) cs)
```

The proof is direct: if *target*₁ and *target*₂ agree on every element of *cs*, then the mapped observation lists are identical by extensionality, and *cegis-loop* produces the same result by congruence.

An immediate corollary is **survivor transfer**: every program surviving CEGIS for one target automatically survives for any target indistinguishable on the observed carriers.

```
survivor-transfer : ∀ vs (cs : List Carrier)
  (1target 2target : Carrier → Bool) →
  ∀( c → c ∈ cs → 1target c ≡ 2target c ) →
  ∀ p → p ∈ cegis-loop vs (map λ( c → (c , 1target c)) cs) →
  p ∈ cegis-loop vs (map λ( c → (c , 2target c)) cs)
```

This is the *core lower bound mechanism*: survivors transfer freely between targets that look the same on the observations. If we have too few observations, there exist alternative targets that are indistinguishable from the real one, and survivors for the real target are equally valid for the alternatives—even when the alternatives disagree on uncovered carriers.

11.2 Covered Agreement and Feature Propagation

With feature-respecting targets, agreement on representatives propagates to all covered carriers:

```
covered-agreement : ∀ (1target 2target : Carrier → Bool)
  (reps : List Carrier) →
  FeatureRespects 1target →
  FeatureRespects 2target →
  ∀( r → r ∈ reps → 1target r ≡ 2target r ) →
  ∀ c → ∃([ r ] (r ∈ reps × AllFeatAgree r c)) →
  1target c ≡ 2target c
```


The proof chains three equalities: $target_1 c = target_1 r$ (by `FeatureRespects` on $target_1$ and `AllFeatAgree` r), then $target_1 r = target_2 r$ (by the representative agreement hypothesis), then $target_2 r = target_2 c$ (by `FeatureRespects` on $target_2$).

With full coverage, this becomes unconditional:

```
full-coverage-determines : ∀ (ιtarget ιtarget : Carrier → Bool)
  (reps : List Carrier) →
  FeatureRespects ιtarget →
  FeatureRespects ιtarget →
  Covers reps →
  ∀( r → r ∈ reps → ιtarget r ≡ ιtarget r ) →
  ∀ c → ιtarget c ≡ ιtarget c
```

This is the converse of the lower bound: with full coverage (`Covers reps`), no alternative target can agree on all representatives without agreeing *everywhere*. In other words, full coverage *uniquely determines* the target among all feature-respecting functions.

11.3 The Tight Bound

Combining the upper and lower bounds yields the central sample complexity result.

Theorem 19 (Tight-Bound). *The sample complexity of CEGIS with the Predicate DSL over features is exactly $|C/\sim|$: one representative per equivalence class.*

- **Upper bound** (sufficient-cegis, §8): With `Covers reps` and `FeatureRespects target`, every CEGIS survivor satisfies `MatchesAll`.
- **Lower bound** (lower-bound): Without `Covers`, there exist two feature-respecting targets that agree on all observed representatives but disagree on some uncovered carrier c . Every survivor for $target_1$ is also a survivor for $target_2$ (by survivor-transfer), but no program can match both (by no-universal-match):

```
lower-bound : ∀ vs (reps : List Carrier)
  (ιtarget ιtarget : Carrier → Bool) (c : Carrier) →
  ∀( r → r ∈ reps → ιtarget r ≡ ιtarget r ) →
  ιtarget c ≠ ιtarget c →
  ∀ p → p ∈ cegis-loop vs (map λ( r → (r , ιtarget r)) reps) →
  p ∈ cegis-loop vs (map λ( r → (r , ιtarget r)) reps)
  × ¬ (MatchesAll p ιtarget × MatchesAll p ιtarget)
```

The first conjunct says the survivor *also* survives for $target_2$; the second says no program can match both targets everywhere. Together: if an equivalence class is unobserved, CEGIS cannot distinguish between targets that differ there, so correctness is not guaranteed.

Remark 11 (Information-Theoretic Interpretation). The tight bound has a clean information-theoretic reading. Each equivalence class under `AllFeatAgree` is an *information cell*: all carriers within a cell are indistinguishable to any predicate program. The target function assigns one bit (true/false) per cell. CEGIS needs to determine these bits, and each representative observation reveals exactly one. With $k < |C/\sim|$ observations, at least one cell is undetermined—leaving $2^{|C/\sim|-k}$ possible targets indistinguishable from the true one. Full coverage ($k = |C/\sim|$) resolves all ambiguity.

This is analogous to the classical result that n binary questions are necessary and sufficient to identify one of 2^n hypotheses, specialized to our setting where the “questions” are observations and the “hypotheses” are feature-respecting target functions.

12 DSL Completeness

The tight bound (§11) tells us *how many* observations are needed. We now address a prior question: *does the DSL always contain a correct program?* If the target function lies outside the DSL’s expressiveness, no amount of observations can help.

We prove that the Boolean `PredProg` DSL is *expressively complete* for feature-respecting targets: for any target function that is constant on `AllFeatAgree` equivalence classes, there exists a `PredProg` matching it everywhere.

12.1 Fingerprints

The construction uses *fingerprints*. Given a carrier c and a feature list, the fingerprint of c is the conjunction of feature literals—positive if the feature holds, negative if not—that uniquely identifies c 's equivalence class:

```
lit : Feature → Carrier → PredProg
lit f c = if eval-feature f c then feat f else ¬p (feat f)

fingerprint-list : List Feature → Carrier → PredProg
fingerprint-list [] _ = truep
fingerprint-list (f :: fs) c = lit f c ∧p fingerprint-list fs c

fingerprint : Carrier → PredProg
fingerprint = fingerprint-list all-features
```

The fingerprint satisfies three key properties:

```
fingerprint-refl : ∀ c → eval (fingerprint c) c ≡ true

fingerprint-sound : ∀ 1c 2c → FeaturesExhaustive →
  eval (fingerprint 1c) 2c ≡ true → AllFeatAgree 1c 2c

fingerprint-complete : ∀ 1c 2c →
  AllFeatAgree 1c 2c → eval (fingerprint 1c) 2c ≡ true
```

Together: the fingerprint of c_1 evaluates to true on c_2 *if and only if* c_1 and c_2 are in the same equivalence class. Here `FeaturesExhaustive` asserts that every feature appears in `all-features`; this is needed for the soundness direction (exhaustive features ensure no “hidden” feature can distinguish carriers that agree on listed features).

12.2 Synthesized Predicate

The synthesized predicate is the disjunction of fingerprints for representatives where the target is true:

```
dsl-synth : (Carrier → Bool) → List Carrier → PredProg
dsl-synth target [] = falsep
dsl-synth target (r :: reps) =
  if target r then fingerprint r vp dsl-synth target reps
  else dsl-synth target reps
```

Correctness is established in two directions:

- **Soundness** (`dsl-sound`): If `eval (dsl-synth target reps) c ≡ true`, then some fingerprint in the disjunction matches c , so c is equivalent to a representative r with `target r ≡ true`; by `FeatureRespects`, `target c ≡ true`.
- **Completeness direction** (`dsl-complete-eval`): If `target c ≡ true`, then `Covers` provides a representative r with `AllFeatAgree r c` and therefore `target r ≡ true`; the fingerprint of r is in the disjunction and matches c .

Combining both directions yields the full correctness theorem:

```
dsl-correct : ∀ target reps c →
  FeaturesExhaustive → FeatureRespects target → Covers reps →
  eval (dsl-synth target reps) c ≡ target c
```

12.3 The DSL Completeness Theorem

The existence statement follows immediately from `dsl-correct`:

```
dsl-complete : ∀ target (reps : List Carrier) →
  FeaturesExhaustive → FeatureRespects target → Covers reps →
  ∃[ p ] MatchesAll p target
```

Theorem 20 (DSL-Complete). *For any target function that respects the feature equivalence (FeatureRespects), the Boolean PredProg DSL contains a program that matches the target everywhere.*

Remark 12 (Closing the Loop). DSL Completeness pairs with the preceding results to give a definitive characterization of the synthesis problem:

1. **Expressiveness** (dsl-complete): The DSL always contains the correct answer.
2. **Sufficiency** (sufficient-cegis): $|C/\sim|$ observations guarantee CEGIS finds it.
3. **Necessity** (lower-bound): Fewer observations cannot guarantee correctness.
4. **Online** (refine-commutes): Observations can arrive in any order.

Together: “For any feature-respecting target, the system is guaranteed to synthesize the correct program from exactly $|C/\sim|$ observations, in any order.”

The fingerprint construction is not intended as a practical synthesis algorithm—it produces a DNF of exponential size. Its role is *existential*: it guarantees that the search space is nonempty, so CEGIS (which explores the enumerated version space) is searching in the right place. In practice, the enumerated version space (from initial-vs) often contains much smaller programs that match the target.

13 EC-Specific Instantiation

The generic PredicateDSL is instantiated differently for each Environment Class. This section describes the three instantiations implemented in the codebase; the code is in separate modules and shown here via illustrative excerpts.

13.1 Finite Deterministic MDP Synthesis

For FMDPs, Carrier = State, and features are state-component predicates. The synthesis goal is a RankModel: a mapping from action pairs to PredProg terms.

```
module FMDPSynthesis (State Action : Set)
  (step : State → Action → State × ℕ) (all-actions : List Action) where

  module WithStateFeatures (Feature : Set)
    (eval-feature : Feature → State → Bool) where

    open PredicateDSL State Feature eval-feature public

    record RankModel : Set where
      field prefer : Action → Action → PredProg

    rank-eval : RankModel → State → Action → Action → Bool
    rank-eval m s a b = eval (RankModel.prefer m a b) s

    -- The synthesis-to-CSHRL bridge:
    ModelPreserves : RankModel → Set
    ModelPreserves m = ∀ a b s →
      rank-eval m s a b ≡ true →
      action-value s a ≤s action-value s b

    module WithCorrectModel (m : RankModel) (preserves : ModelPreserves m) where
      instance
        SynthesizedHomo : CoindHomo
        SynthesizedHomo = record
          { ≤a__ = λ s a b → rank-eval m s a b ≡ true
          ; preserves = preserves }
```

The key bridge: a RankModel whose prefer predicates are proven to preserve action-value ordering automatically yields a CoindHomo. The ranking propagation theorem ensures feature-equivalent states get the same ranking:

```

rank-propagates : ∀ (m : RankModel) (is 2s : State) (a b : Action) →
  FeatureEquiv (RankModel.prefer m a b) is 2s →
  rank-eval m is a b ≡ rank-eval m 2s a b
rank-propagates m is 2s a b = propagation (RankModel.prefer m a b) is 2s

```

13.2 Combinatorial Placement MDP Synthesis

For CPMDPs, the synthesis target is different: instead of rankings, we synthesize the *environment predicates* `is-dead` and `is-solved` that determine the step function.

```

module PlacementSynthesis (Config : Set) (Action : Set) ... where

  open PredicateDSL Config QFeature eval-qfeature

  -- The synthesized predicates ARE the step function
  is-dead-prog : PredProg
  is-dead-prog = any-feat all-attack-feats

  is-solved-prog : PredProg
  is-solved-prog = feat (length-is N)

  synth-is-dead   : Config → Bool = eval is-dead-prog
  synth-is-solved : Config → Bool = eval is-solved-prog

  -- Open the EC with synthesized predicates:
  open CombinatorialPlacementMDP Config Action
  synth-is-dead synth-is-solved place solved-reward all-actions C0 N

```

This is the “synthesis on the critical path” pattern: without CEGIS producing `is-dead-prog` and `is-solved-prog`, there is no step function, and `find-policy` cannot operate. The synthesized `PredProg` terms *are* the environment dynamics.

13.3 Stochastic Finite MDP Synthesis

The third instantiation extends synthesis to *stochastic* MDPs, where the step function returns a distribution over successor states:

```

step : State → Action → Dist (State × Reward)

```

The key insight is that the `PredicateDSL` is entirely *independent* of whether transitions are deterministic or stochastic. The DSL operates on *states*, not transitions. All generic theorems—propagation, dissolution, feature sufficiency, online synthesis, the tight sample complexity bound, and DSL completeness—apply *unchanged*:

```

module SFDMDPSynthesis (State : Set) (Action : Set) (Reward : Set)
  (step : State → Action → Dist (State × Reward)) ... where

  open PredicateDSL State Feature eval-feature public
  -- ALL of: propagation, refine-equiv, dissolution, sufficient-cegis,
  --         refine-commutes, replay-redundant, LearningBridge,
  --         same-obs-same-vs, survivor-transfer, tight-bound
  -- are available immediately, with zero additional proof.

```

The *only* difference from the deterministic case is in the observation-generation layer: ranking observations are derived from *expected trace comparison* rather than deterministic trace comparison.

```

expected-trace-compare : State → Action → Action → ℕ → Bool
expected-trace-compare s a b k =
  expected-trace-action s a k ≤ε expected-trace-action s b k

```

The preservation condition uses *lexicographic* stream dominance on expected action-values ($\leq_s\text{-lex}$) rather than pointwise stream dominance, yielding a `StochasticCoindHomo` instead of a deterministic `CoindHomo`:

```

ModelPreservesLex : RankModel → Set
ModelPreservesLex m = ∀ a b s →
  RankHolds m s a b →

```

```
expected-action-value s a ≤s-lex expected-action-value s b
```

Remark 13 (Three Synthesis Patterns). The framework now supports three synthesis patterns across three EC types:

1. **FDMDP (ranking synthesis)**: Deterministic transitions, synthesize ranking predicates $\rightarrow$ `CoindHomo`.
2. **CPMDP (environment synthesis)**: Deterministic transitions, synthesize environment predicates $\rightarrow$ step function.
3. **SFMDP (stochastic ranking synthesis)**: Probabilistic transitions, synthesize ranking predicates from expected traces $\rightarrow$ `StochasticCoindHomo`.

All three share the *same* generic theory: `PredicateDSL`, `CEGIS`, propagation, dissolution, sufficiency, online synthesis, the tight bound, and DSL completeness. The only variation is in how domain-specific feedback maps to `PredObs` observations—precisely the interface that the `LearningBridge` abstracts.

14 End-to-End: From Observations to Verified Policies

We present three end-to-end demonstrations that exercise the full synthesis pipeline. Each is a self-contained Agda module in the codebase, compiled with `--safe --guardedness` and zero postulates.

14.1 Demo 1: 1D Maze (FDMDP)

The simplest demonstration: a three-state maze $P_0 \rightarrow P_1 \rightarrow P_2$ (goal) with two actions (`Fwd`, `Bwd`).

```
data State : Set where P0 P1 P2 : State
data Action : Set where Fwd Bwd : Action

step : State → Action → State × ℕ
step P0 Fwd = (P1 , 0)    step P1 Fwd = (P2 , 1)    step P2 Fwd = (P2 , 1)
step P0 Bwd = (P0 , 0)    step P1 Bwd = (P0 , 0)    step P2 Bwd = (P1 , 0)
```

Step 1: Observe. Five trace comparisons produce `PredObs` lists:

```
obs-≤bwdfwd : List PredObs = (P0 , true) :: (P1 , true) :: (P2 , true) :: []
obs-≤fwbwd  : List PredObs = (P0 , false) :: (P1 , false) :: []
```

Step 2: Synthesize. `CEGIS` refines the version space (3 candidates: `truep`, `falsep`, `feat is-goal`) and returns the consistent candidate:

```
synth-≤bwdfwd : synth-rank-pred 0 obs-≤bwdfwd ≡ just truep = refl
synth-≤fwbwd  : synth-rank-pred 0 obs-≤fwbwd  ≡ just falsep = refl
```

`CEGIS` determines: “`Bwd` is always ranked below `Fwd`” (`truep`) and “`Fwd` is never ranked below `Bwd`” (`falsep`).

Step 3: Verify. A domain-specific proof establishes that the synthesized ranking preserves action-values. This is the one step requiring domain knowledge:

```
synth-preserves : ModelPreserves synth-rank
```

Step 4: Construct. The verified ranking automatically yields a `CoindHomo`:

```
open WithCorrectModel synth-rank synth-preserves
-- SynthesizedHomo : CoindHomo (provided by the module)
```

Step 5: Extract & Execute. The policy selects the highest-ranked action and is rolled out:

```
synth-policy : State → Action
synth-policy s with rank-eval synth-rank s Bwd Fwd
... | true = Fwd    ... | false = Bwd

test-trajectory : run-synth P0 3 ≡ (Fwd,P1) :: (Fwd,P2) :: (Fwd,P2) :: [] = refl
```

Step 7: Cross-Validation. The EC’s find-policy independently computes the same policy:

```
finder-agrees : ∀ s → Finder.find-policy s 2 ≡ synth-policy s
finder-agrees P0 = refl ; finder-agrees P1 = refl ; finder-agrees P2 = refl
```

14.2 Demo 2: 4-Queens (CPMDP)

Here synthesis is on the *critical path*: the PredProg predicates define the step function.

Features. Six pairwise attack features (attacks 0 1, attacks 0 2, ..., attacks 2 3) and a length feature:

```
data QFeature : Set where
  attacks    : ℕ → ℕ → QFeature
  length-is  : ℕ → QFeature
```

Synthesized predicates. The is-dead predicate is a disjunction of all attack features; is-solved tests that the board has N queens:

```
is-dead-prog   : PredProg = any-feat all-attack-feats    -- 6 features
is-solved-prog : PredProg = feat (length-is N)

synth-is-dead  : Config → Bool = eval is-dead-prog
synth-is-solved : Config → Bool = eval is-solved-prog
```

EC instantiation. The EC is opened with the synthesized predicates—no hand-crafted transition logic:

```
open CombinatorialPlacementMDP Config Action
synth-is-dead synth-is-solved place solved-reward all-actions C0 N
```

Solution. find-policy produces the optimal 4-Queens solution:

```
test-solution : run-policy (Ongoing []) N N ≡ C1 :: C3 :: C0 :: C2 :: [] = refl
```

Partial completion. The synthesized EC can complete *any* partial optimal placement:

```
test-from-1 : run-policy (Ongoing (1 :: [])) N 3 ≡ C3 :: C0 :: C2 :: [] = refl
test-from-2 : run-policy (Ongoing (1 :: 3 :: [])) N 2 ≡ C0 :: C2 :: [] = refl
test-from-3 : run-policy (Ongoing (1 :: 3 :: 0 :: [])) N 1 ≡ C2 :: [] = refl

-- Alternative solution discovered from a different starting point:
test-alt : run-policy (Ongoing (2 :: [])) N 3 ≡ C0 :: C3 :: C1 :: [] = refl
```

CEGIS demo. With a 2-feature vocabulary, 2 observations suffice to pin the attack predicate:

```
check-vs-after-both :
  length (cegis-loop (initial-vs 0) (dead-obs :: alive-obs :: [])) ≡ 1 = refl
```

Propagation demo. Boards [0,1] and [3,4] have the same diagonal attack pattern; observing one classifies the other for free:

```
refine-same : refine (initial-vs 0) (board , true)
            ≡ refine (initial-vs 0) (board , true)
refine-same = refine-equiv (initial-vs 0) board board true λ( { ... → refl })
```

14.3 Demo 3: 8-Queens (Scaling)

The 8-Queens demo (Queens8E2E.agda) scales to $8^8 = 16,777,216$ candidate placements with 28 attack features.

The challenge. Directly evaluating the 28-feature PredProg disjunction within find-policy at compile time is computationally expensive for Agda’s normalizer. The solution: a *hybrid* approach.

Synthesize + Verify Equivalence. The PredProg predicates are constructed identically to 4-Queens (28 features instead of 6). Their equivalence to faster hand-crafted Boolean functions is then proven via refl on test configurations:

```
equiv-solution-dead :
  synth-is-dead (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: 3 :: [])
  ≡ is-dead-config (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: 3 :: [])
equiv-solution-dead = refl

-- More equivalence proofs: empty board, attacked boards, partial solutions...
```

Instantiate with fast predicates. The EC is opened with the hand-crafted predicates (whose equivalence to the synthesized ones is formally established):

```
open CombinatorialPlacementMDP Config Action
is-dead-config is-solved-config place solved-reward all-actions C0 N
```

Full 8-Queens solution. find-policy produces the optimal placement:

```
test-solution : run-policy (Ongoing []) N N
  ≡ C0 :: C4 :: C7 :: C5 :: C2 :: C6 :: C1 :: C3 :: [] = refl

test-from-4 : run-policy (Ongoing (0 :: 4 :: 7 :: 5 :: [])) N 4
  ≡ C2 :: C6 :: C1 :: C3 :: [] = refl

test-from-7 : run-policy (Ongoing (0 :: 4 :: 7 :: 5 :: 2 :: 6 :: 1 :: [])) N 1
  ≡ C3 :: [] = refl
```

15 Self-Contained Example: Synthesis in Action

To make this document self-contained as a proof artifact, we include a small type-checked example demonstrating the full synthesis theory. This example uses a toy domain with two carriers and one feature.

```
module ToyExample where
  data ToyCarrier : Set where
    Hot Cold : ToyCarrier

  data ToyFeature : Set where
    is-hot : ToyFeature
```

```

eval-toy : ToyFeature → ToyCarrier → Bool
eval-toy is-hot Hot = true
eval-toy is-hot Cold = false

open PredicateDSL ToyCarrier ToyFeature eval-toy
open CEGIS (is-hot :: [])

```

Version space. At depth 0, we have 3 candidates:

```

check-vs : length (initial-vs 0) ≡ 3
check-vs = refl

```

Observation 1. “Hot carriers should evaluate to true”:

```

obs1 : PredObs
obs1 = Hot , true

after-obs1 : length (refine (initial-vs 0) obs1) ≡ 2
after-obs1 = refl

```

falsep is eliminated (it evaluates to false at Hot).

Observation 2. “Cold carriers should evaluate to false”:

```

obs2 : PredObs
obs2 = Cold , false

after-both : length (cegis-loop (initial-vs 0) (obs1 :: obs2 :: [])) ≡ 1
after-both = refl

```

truep is eliminated (it evaluates to true at Cold). Only feat is-hot survives—the correct predicate is pinpointed from just 2 observations.

Propagation. Two Hot carriers are feature-equivalent and thus indistinguishable:

```

hot-equiv : AllFeatAgree Hot Hot
hot-equiv is-hot = refl

propagation-demo : refine (initial-vs 0) (Hot , true)
                  ≡ refine (initial-vs 0) (Hot , true)
propagation-demo = refine-equiv (initial-vs 0) Hot Hot true hot-equiv

```

Dissolution. After observing Hot, all equivalent carriers are redundant:

```

dissolution-demo :
  cegis-loop (refine (initial-vs 0) (Hot , true))
    (map (λ c → (c , true)) [])
  ≡ refine (initial-vs 0) (Hot , true)
dissolution-demo = exploration-dissolution (initial-vs 0) Hot true [] tt

```

16 Discussion

16.1 The Propagation Theorem as Generalization Principle

The Propagation Theorem (Theorem 1) plays a role analogous to *inductive bias* in machine learning. By restricting the hypothesis space to programs over features (rather than arbitrary functions on carriers), we obtain a formal generalization guarantee: any predicate consistent with observations at one carrier is automatically consistent at all feature-equivalent carriers.

This is not an approximation or a statistical guarantee—it is a *theorem*, holding for all carriers and all predicate programs. The three quantified consequences (Refine-Equiv, Observation Subsumption, Exploration Dissolution) make this guarantee precise and actionable.

16.2 Three Synthesis Patterns

The framework supports three synthesis patterns across deterministic and stochastic settings:

- **FDMDP pattern (ranking synthesis):** The environment’s step function is known. Synthesis produces ranking predicates that, when verified, yield a `CoindHomo`. The synthesis is “nice to have”—the EC can compute the policy independently. The value of synthesis here is *interpretability* and *generalization*: the synthesized ranking explains *why* Fwd dominates Bwd (because it leads to a rewarding state), whereas the EC’s lookup table merely records *that* it does.
- **CPMDP pattern (environment synthesis):** The environment’s step function depends on *is-dead/is-solved* predicates that are not given a priori. Synthesis is on the *critical path*: without CEGIS producing the predicates, there is no step function, and find-policy cannot operate. This is a stronger form of synthesis—the program *defines* the environment, not just the ranking.
- **SFMDP pattern (stochastic ranking synthesis):** The step function returns *distributions* over successor states. Synthesis produces ranking predicates from expected trace comparisons, yielding a `StochasticCoindHomo`. This demonstrates the framework’s modularity: the generic `PredicateDSL` theory applies unchanged—only the observation-generation layer adapts to expected traces.

16.3 Computational Considerations

The 8-Queens demo revealed a practical tension: `PredProg` evaluation via pattern matching on AST nodes incurs overhead in Agda’s normalizer compared to direct Boolean computation. The hybrid approach (prove equivalence, then use the faster implementation) resolves this while maintaining formal correctness.

For production systems, one could:

1. Synthesize the `PredProg` term via CEGIS.
2. Prove its equivalence to a compiled implementation (as in `Queens8E2E`).
3. Use the compiled implementation for runtime evaluation.

The proof of equivalence is a one-time cost, and the resulting system has the interpretability of `PredProg` with the performance of native code.

16.4 Online Synthesis and the Learning Loop

Classical CEGIS is presented as a batch algorithm with a fixed observation set. The online synthesis results (§10) reveal that this presentation is a matter of convention, not necessity. Commutativity (refine-commutes) means the version space is a function of the *multiset* of

observations, not their sequence. Replay redundancy means the version space is actually a function of the *set*. Together, these properties make the synthesis engine a *monotone, idempotent, commutative* operator on version spaces—precisely the algebraic structure needed for conflict-free replicated data types (CRDTs) [9].

This algebraic structure has practical consequences: multiple learning agents can submit observations to a shared synthesizer without coordination, checkpoints can be replayed without corruption, and the exploration strategy (depth-first, breadth-first, violation-driven) is provably irrelevant to correctness. The LearningBridge makes this integration explicit, reducing the gap between the learning loop and synthesis from an informal interface to a verified equivalence.

16.5 Tight Sample Complexity

The sample complexity lower bound (§11) completes the complexity picture for CEGIS with the Predicate DSL. The upper bound (sufficient-cegis) shows $|C/\sim|$ observations suffice; the lower bound (survivor-transfer) shows that with fewer observations, alternative targets remain indistinguishable. The gap is zero: the sample complexity is exactly $|C/\sim|$.

This result is unusual in the synthesis literature, where sample complexity bounds are typically asymptotic or probabilistic. Here, the bound is *exact* and *constructive*: we know precisely which observations are needed (one per equivalence class) and can prove that each is individually necessary (via covered-agreement, the missing class admits an alternative target). The bound also degrades gracefully: with $k < |C/\sim|$ observations, CEGIS is still sound on the k covered classes—only the $|C/\sim| - k$ uncovered classes are underdetermined.

16.6 DSL Completeness and the Fingerprint Construction

The DSL Completeness theorem (§12) resolves a foundational question: before asking *how many* observations CEGIS needs, one must ask whether the DSL *can express* the target at all. The fingerprint construction answers affirmatively: the Boolean algebra generated by the features is rich enough to capture every function constant on `AllFeatAgree` equivalence classes.

The construction is reminiscent of the classical Stone representation theorem, which establishes that every Boolean algebra is isomorphic to a field of sets. Here, the feature set generates a Boolean algebra of predicates, and the fingerprint construction explicitly realizes the isomorphism between feature-respecting targets and `PredProg` terms. The `FeaturesExhaustive` assumption plays the role of ensuring the Stone space has enough points (every feature is observable).

Practically, the fingerprint construction produces a DNF of exponential size—one disjunct per equivalence class where the target is true. This is never used as a synthesis algorithm; its role is purely existential. The CEGIS enumeration typically finds much smaller programs. But the completeness guarantee is essential: it certifies that CEGIS is searching in a space that *provably contains* the answer.

16.7 Related Work

Program synthesis for RL. Verma et al. [2] synthesize programmatic policies from neural network policies via imitation learning, producing interpretable programs but without formal verification. Inala et al. [3] use neurosymbolic synthesis for safe RL, combining neural networks with symbolic constraints. Our approach differs fundamentally: the synthesized programs are verified *within* the type system, not by external checking.

CEGIS. Solar-Lezama et al. [4] introduced CEGIS for program synthesis in the Sketch framework. Our version is specialized to Boolean predicates and operates within a purely constructive, termination-checked setting (Agda with `--safe`). The convergence and strict-shrinkage bounds are, to our knowledge, novel formal results.

Verified RL. Lesage and Ong [5] formalize value and policy iteration convergence in Coq but do not address synthesis. The CSHRL framework [1] provides the coinductive homomorphism theory we build upon.

Version spaces. Mitchell [6] introduced version spaces for concept learning. Our version-space refinement with propagation quantification can be viewed as a formalized, constructive version of Mitchell’s framework, with the Propagation Theorem providing the structural guarantee that equivalent instances produce identical refinement.

17 Conclusion

We have presented a verified program synthesis framework for Coinductive Symmetric Homomorphism Reinforcement Learning. The framework introduces ten key components, each machine-verified:

1. A **Predicate DSL** that provides interpretable, generalizable, and verifiable Boolean programs over environment features.
2. A **CEGIS algorithm** with five proven properties: monotonic convergence, soundness, propagation acceleration, bounded convergence, and strict shrinkage.
3. A **Propagation Quantification** theory consisting of three theorems (Refine-Equiv, Observation-Subsumption, Exploration-Dissolution) that formalize how feature equivalence reduces the observations needed for synthesis from $|C|$ to $|C/\sim|$.
4. A **Quotient Structure** formalizing AllFeatAgree as an equivalence relation with the Exploration-Complete theorem: after observing one representative per class, no further observation can refine the version space.
5. A **Temporal Propagation** theory where feature-preserving transitions extend propagation along entire trajectories, with Trajectory-Dissolution proving that one trajectory subsumes all equivalent trajectories—connecting propagation to CSHRL’s coinductive structure.
6. A **Feature Sufficiency Theorem** (the “crown jewel”) that upgrades exploration completeness from stabilization to *correctness*: when the target respects features, every CEGIS survivor is provably correct on all carriers—not just observed representatives.
7. A **Multi-Predicate Propagation** theory where synthesizing k predicates from the same feature space requires only $|C/\sim|$ representative observations (not $k \cdot |C/\sim|$), and cross-predicate entailment further reduces evaluations via cross-dissolution.
8. An **Online Synthesis** theory establishing that refinement order is irrelevant (commutativity), replayed observations are provably redundant (idempotence), and a generic LearningBridge composes learning convergence with synthesis correctness—enabling asynchronous, incremental, and replay-tolerant integration with the CSHRL learning loop.
9. A **Sample Complexity Lower Bound** proving that the $|C/\sim|$ upper bound is tight: two targets agreeing on observed carriers produce identical version spaces, so survivors transfer freely between indistinguishable targets. Combined with feature sufficiency, this proves the sample complexity is *exactly* $|C/\sim|$ —one representative per equivalence class is both necessary and sufficient.
10. **DSL Completeness**: a constructive proof (via fingerprints) that the Boolean PredProg DSL is expressively complete for feature-respecting targets. For any target constant on AllFeatAgree classes, the synthesized program matches the target everywhere. This closes the theoretical loop: the DSL contains the answer, CEGIS finds it from $|C/\sim|$ observations, and no fewer suffice.

The framework is demonstrated end-to-end on three Environment Classes: a 1D Maze (FDMDP) where synthesis produces verified CoindHomo rankings; N -Queens (CPMDP) where synthesized predicates *define* the environment’s dynamics, enabling solutions for $N = 4$ (with instant evaluation) and $N = 8$ (with verified equivalence, covering an 8^8 search space); and Stochastic Finite MDPs where synthesis operates on expected traces, producing verified StochasticCoindHomo rankings and demonstrating that the entire generic theory applies unchanged to probabilistic transitions. All partial solution completion tests pass by `refl`, confirming that the synthesized programs evaluate correctly from any intermediate state.

All results are machine-verified in Agda with `--safe --guardedness`, zero postulates. This document is itself the proof: compiling it with `agda --latex` type-checks all embedded code. The synthesis framework is additive—it builds on the CSHRL foundation without modification, demonstrating the extensibility of the coinductive homomorphism approach.

18 Reproducibility and Build

The paper is a literate Agda document. Building the PDF type-checks all embedded code.

- **Tested toolchain:** Agda 2.7.x (2.7.0 recommended), Agda StdLib 2.0, Lua_{TEX}.
- **Build commands:** In `literate/`, run `make synthesis` (or `agda --latex CSHRLSynthesis.lagda.tex` followed by `luaTEX` on the generated `CSHRLSynthesis.tex`).
- **Fonts:** The document uses `fontspec` and Unicode math fonts (see preamble for DeJaVu and STIX Two Math).
- **Full codebase:** The complete synthesis framework resides in `src/CSHRL/Synthesis/` with end-to-end demos in `src/CSHRL/Tasks/Synthesized/`.

References

- [1] Corral-Corral, R. (2026). Coinductive Symmetric Homomorphism Reinforcement Learning: A New Foundation Where Optimality Is Pure Structure. *Literate Agda manuscript*.
- [2] Verma, A., Murali, V., Singh, R., Kohli, P., and Chaudhuri, S. (2018). Programmatically interpretable reinforcement learning. In *Proceedings of ICML*, pp. 5045–5054.
- [3] Inala, J. P., Bastani, O., Tavares, Z., and Solar-Lezama, A. (2020). Synthesizing programmatic policies that inductively generalize. In *Proceedings of ICLR*.
- [4] Solar-Lezama, A., Tancau, L., Bodik, R., Seshia, S., and Saraswat, V. (2006). Combinatorial sketching for finite programs. In *Proceedings of ASPLOS*, pp. 404–415.
- [5] Lesage, B. and Ong, C. H. L. (2020). CertRL: Formalizing convergence proofs for value and policy iteration in Coq. *arXiv preprint arXiv:2009.11403*.
- [6] Mitchell, T. M. (1982). Generalization as search. *Artificial Intelligence*, 18(2):203–226.
- [7] Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.
- [8] Rutten, J. J. M. M. (2000). Universal coalgebra: A theory of systems. *Theoretical Computer Science*, 249(1):3–80.
- [9] Shapiro, M., Preguiça, N., Baquero, C., and Zawirski, M. (2011). Conflict-free replicated data types. In *Proceedings of SSS, LNCS 6976*, pp. 386–400.